

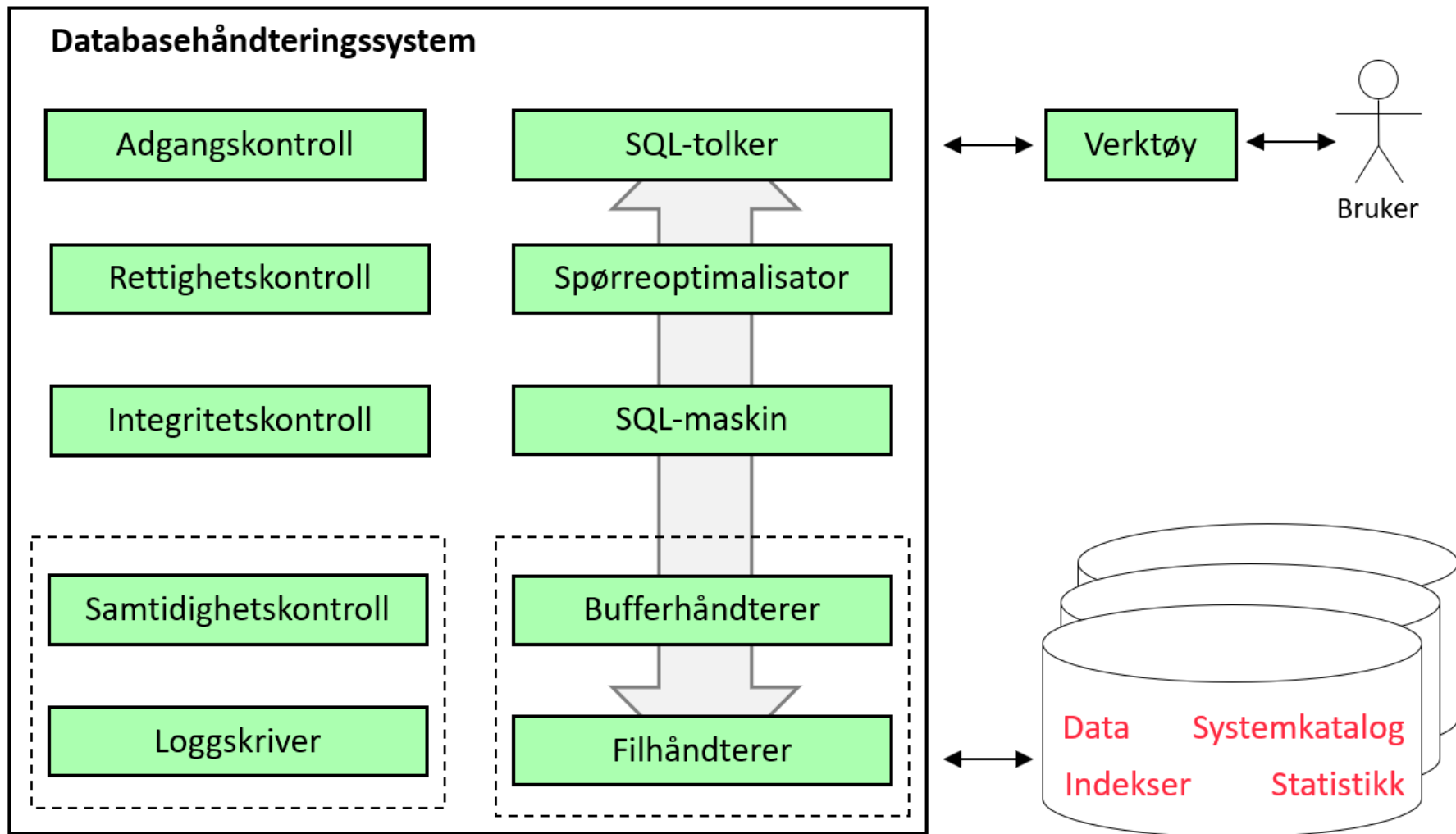
Kapittel 11

Databaseadministrasjon

Læringsmål

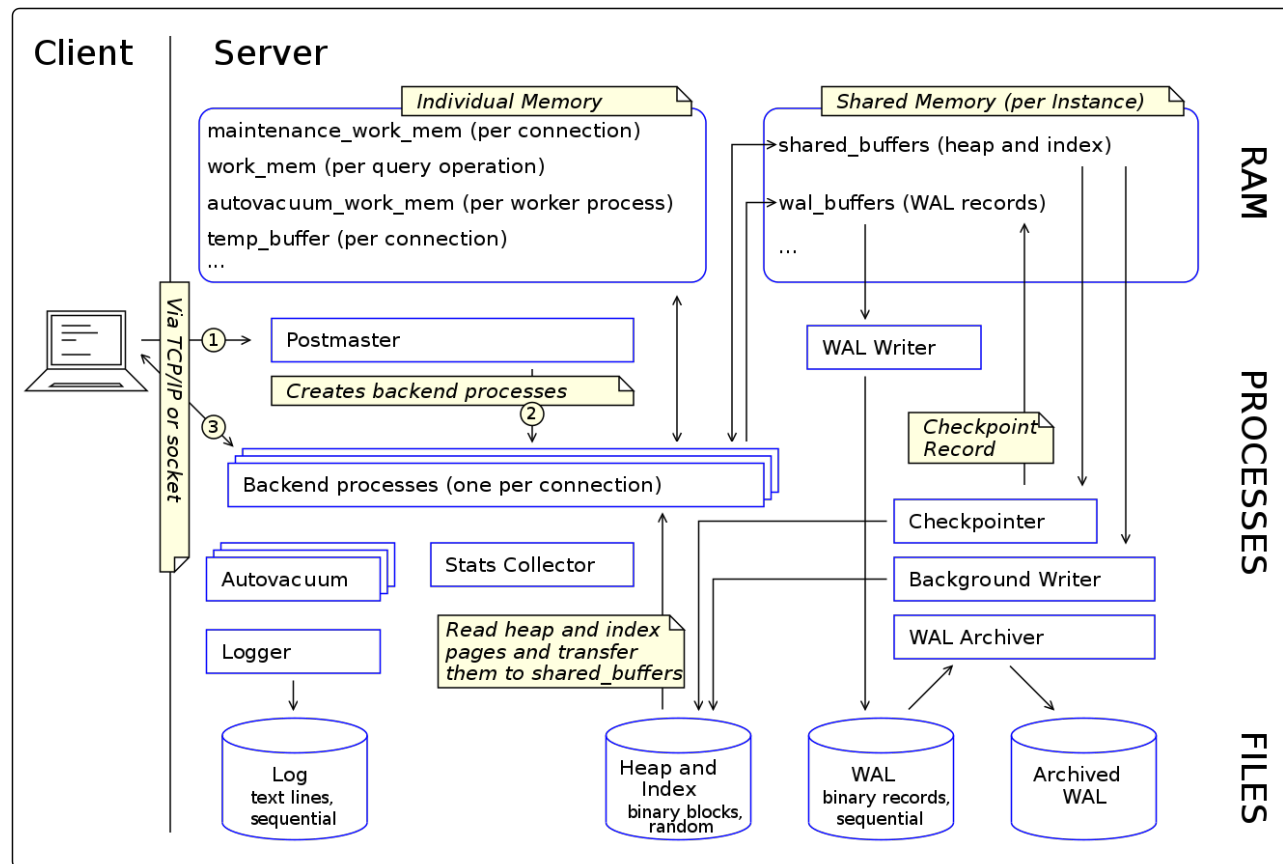
- Kjenne til prinsipiell oppbygging av et database-håndteringssystem (DBMS) og hvilke oppgaver det utfører.
- Kunne utføre enkel databaseadministrasjon, bla. Brukeradministrasjon.
- Kjenne til viktige begreper innen systemarkitekturer for databaseløsninger, som klient/tjener, parallelle databaser, distribuerte databaser og minnedatabaser.
- Forstå fordeler og ulemper ved skydatabaser.

Tenkt oppbygging av et DBHS



PostgreSQL Architecture

Postmaster works as the supervisor process, and it is the first process that gets started after PostgreSQL starts. It is responsible for authenticating and authorizing the incoming request from the client and assigning a new process called **Postgres** for each connection.



Shared memory and background processes for managing system resources efficiently and performing tasks such as logging, vacuuming, and transaction management.

Oppgavene til DBA

- Man skiller gjerne mellom **dataadministrasjon** (strategisk) og **databaseadministrasjon** (teknisk).
- Definere retningslinjer og prosedyrer
- Evaluering, valg og installasjon av DBHS og andre verktøy
- **Brukeradministrasjon**
- Overvåking
- Opplæring og brukerstøtte
- Sikkerhetskopiering (backup) og gjenoppbygging (recovery) etter feil
- Kapasitetsplanlegging og optimalisering

DBA-verktøy

- Mange av DBA-oppgavene kan gjøres med SQL.
- Finnes også egne «DBA-verktøy». Typisk funksjonalitet:
 - Starte og ta ned databasen
 - Sikkerhetskopiering og gjenoppbygging etter feil
 - Definere tabeller, indekser, ...
 - Brukeradministrasjon
 - Sette grenser for brukernes ressursbruk
 - Organisere fysiske lagringsstrukturer
 - Skrive og analysere SQL-kommandoer
 - Visualisere bruk av disk, buffer
 - Gjøre databasen tilgjengelig i et nettverk

Sikkerhetstrusler og virkemidler

■ Uhell

- Menneskelig svikt
- Programvarefeil
- Maskinvarefeil
- Strømstans

■ Angrep

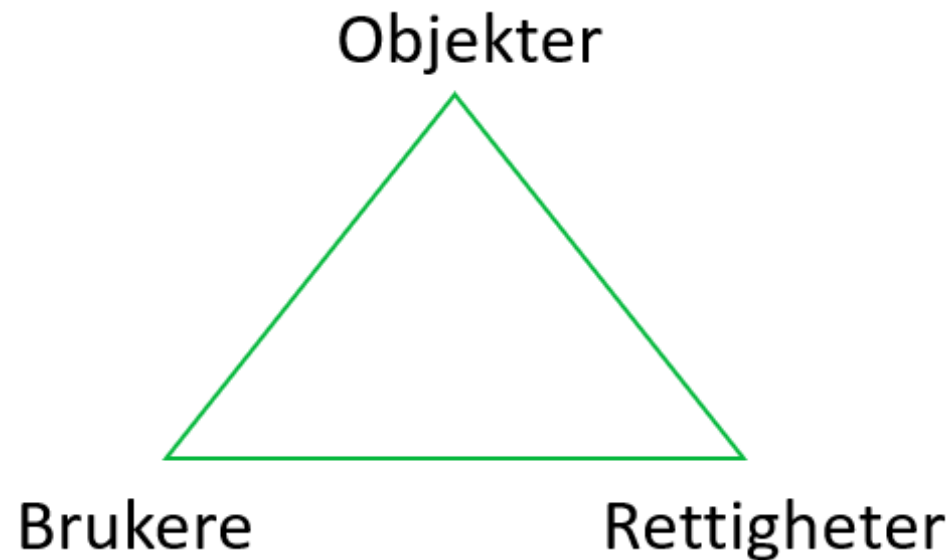
- Ansatte / eksterne
- Innsyn / endring
- Direkte mot databasen
- Indirekte via operativsystem / nettverk

■ Virkemidler

- Sikkerhetskopiering, loggføring og gjenoppbygging
- Overvåking
- Kryptering
- Aksesskontroll
 - Fysisk kontroll
 - Passord
 - Rettigheter
- Duplisering av utstyr
- Nøddaggregat
- Rutiner, «brannøvelser»

Brukeradministrasjon

- Vi ønsker å tildele en bestemt **bruker** retten til å utføre en bestemt **operasjon** på et bestemt **objekt**.
 - Eksempel: Per skal kunne lese kundetabellen.



- Vi skiller mellom **objektrightigheter** (privilegium) og **systemrettigheter**.
 - Starte og stoppe DBHS er eksempel på en systemrettighet.
- Brukerinformasjonen blir lagret i en eller flere **systemtabeller**.

Roller / grupper

- En **rolle** (eller **gruppe**) er en samling brukere som skal ha de samme rettighetene i databasesystemet.
 - Eksempel: Rollen Selger
- Bruk av roller:
 - Opprett roller
 - Tildel roller rettigheter på ulike objekter
 - Opprett brukere
 - Gi brukere roller
- Brukere **arver** rettigheter fra rollene de har.
- DBA slipper dermed å tildele enkeltbrukere rettigheter.

Et grovmasket sikkerhetssystem

- Vi skiller kun mellom noen få brukertyper.
 - **Administratorer**: Får alle rettigheter, også systemrettigheter som å opprette andre brukere, starte og stoppe databasen, lage tabeller osv.
 - **Endre**: Sluttbrukere som både kan avlese og endre innholdet av databasen. Får rettigheter til å avlese / sette inn / oppdatere og slette data (SELECT + INSERT + UPDATE + DELETE).
 - **Innsyn**: Får rettigheter til å avlese innholdet i databasen (SELECT).
 - **Utviklere**: Jobber med strukturen. Får rettigheter til å lage nye tabeller, spørringer, utsnitt, og også å endre definisjonen av eksisterende objekter. Jobber vanligvis ikke med databaser i produksjon.
- Her skiller vi altså ikke mellom konkrete objekter.

Rettigheter i SQL

- Per skal kunne lese og oppdatere tabellen Kunder:
GRANT SELECT, UPDATE
ON Kunder
TO Per
- Per skal ikke lenger få lov til å oppdatere tabellen Kunder:
REVOKE UPDATE
ON Kunder
FROM Per
- For å få lov til å gi rettigheten videre, føy til:
WITH GRANT OPTION - for objektrettigheter
WITH ADMIN OPTION - for systemrettigheter

Et mer finmasket sikkerhetssystem

- Forskjellige deler av databasen blir brukt av forskjellige avdelinger/stillingskategorier.
- Vi deler inn gruppen av sluttbrukere. Eksempler:
 - AvdelingA, AvdelingB, ...
 - Sekretær, Selger, Direktør, ...
 - Nivå1, Nivå2, Nivå3, ...
- Kan være nyttig å lage roller i flere «lag».
 - I Oracle kan roller tildeles andre roller. Eksempel: Både rollen Selger og rollen Sekretær trenger rollen Innsyn.
 - Hvis Selger og Sekretær skal ha mange av de samme rettighetene er det arbeidsbesparende å tildele rettighetene 1 gang til Innsyn, og deretter gi Innsyn til Selger og Sekretær.
- Kan tildele rettigheter på **views** som gir tilgang på utvalgte rader / kolonner i en tabell.

Eksempel: SQL og metadata i PostgreSQL

- Hvordan er tabellene mine definert?

```
SELECT table_name,  
       column_name,  
       data_type  
FROM information_schema.columns  
WHERE table_schema = 'public';
```

- Hvilke rettigheter er delt ut på tabellene til Per:

```
SELECT  
    table_schema,  
    table_name,  
    privilege_type,  
    is_grantable  
FROM information_schema.role_table_grants  
WHERE grantee = 'Per'  
ORDER BY table_schema, table_name, privilege_type;
```

- Hvilke rettigheter har Per delt ut:

```
SELECT table_schema,  
       table_name,  
       grantee,  
       privilege_type,  
       is_grantable  
FROM information_schema.role_table_grants  
WHERE table_schema = 'public'  
      AND grantor = 'Per'  
ORDER BY table_name, grantee, privilege_type;
```

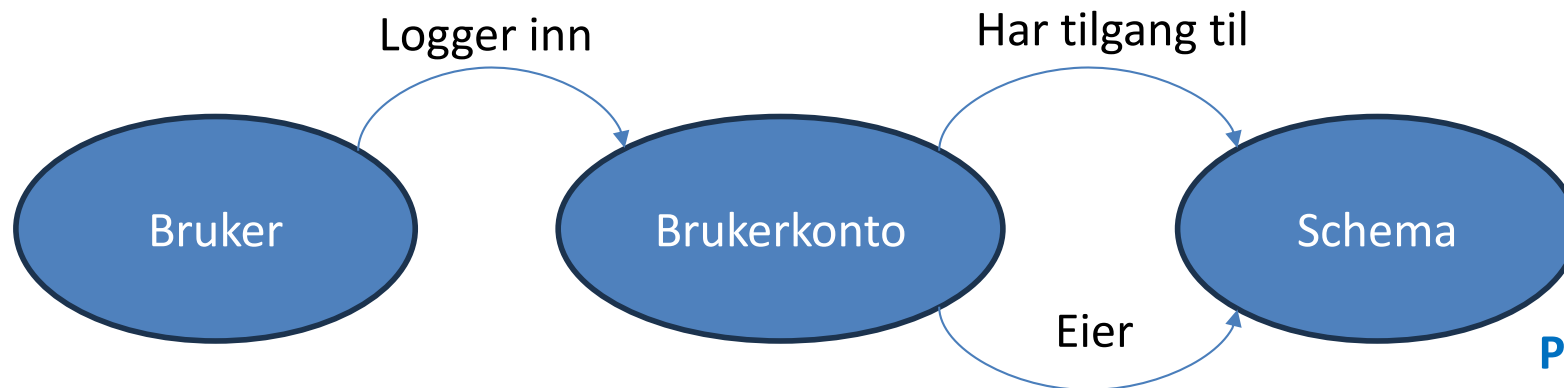
Systemkatalogen

- **Metadata**, eller «data om data» er beskrivelser av databasen:
 - Hvilke tabeller finnes? Hvordan er tabellene bygget opp?
 - Hvilke indekser er definert?
- **Statistiske data** brukes av spørreoptimalisatoren:
 - Hvor mange rader har tabellene?
 - Hvor mange forskjellige verdier er lagret i en kolonne?
- Data om **brukere** og deres **rettigheter**.
- **Systemkatalogen** er en samlebetegnelse slike data.
- Data i systemkatalogen er ofte lagret på tabellform (systemtabeller), som betyr at vi kan bruke SQL for rapportering.

Databasesikkerhet

- Databasesikkerhet er en samling av sikkerhetspolicies og -prosedyrer, databegrensninger, sikkerhetsmetoder og sikkerhetsverktøy som er blandet sammen for å implementere alle nødvendige tiltak for å sikre konfidensialitet, integritet og tilgjengelighet til alle komponenter i databasemiljøet.
- Disse komponentene inkluderer personer, applikasjoner, nettverk, operativsystemer, databasehåndteringssystemer, datafiler og data.

Tilgang til og Bruk av Database Objekter



Person «Joe»
eller
applikasjon

Brukernavn og passord
Autentisering
Rolle
Konto status

Tabeller
Indekser
Prosedyrer
Triggere
osv.

PostgreSQL representerer brukerkontoer som roller. Roller som kan logge inn kalles innloggingsroller eller brukerroller. Roller som inneholder andre roller kalles grupperoller.

▼ Login/Group Roles (15)

- pg_checkpoint
- pg_create_subscription
- pg_database_owner
- pg_execute_server_program
- pg_monitor
- pg_read_all_data
- pg_read_all_settings
- pg_read_all_stats
- pg_read_server_files
- pg_signal_backend
- pg_stat_scan_tables
- pg_use_reserved_connectic
- pg_write_all_data
- pg_write_server_files
- postgres

PostgreSQL

▼ Schemas (1)

- public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - 1.3 Sequences
 - Tables
 - Trigger Functions
 - Types
 - Views

Eksempel på Oppretting av Brukerrolle

- **En rolle** er en entitet som kan eie databaseobjekter og ha databaserettigheter; en rolle kan betraktes som en «bruker», en «gruppe» eller begge deler, avhengig av hvordan den brukes.

```
CREATE ROLE name [ [ WITH ] option [ ... ] ]
```

where *option* can be:

```
SUPERUSER | NOSUPERUSER  
| CREATEDB | NOCREATEDB  
| CREATEROLE | NOCREATEROLE  
| INHERIT | NOINHERIT  
| LOGIN | NOLOGIN  
| REPLICATION | NOREPLICATION  
| BYPASSRLS | NOBYPASSRLS  
| CONNECTION LIMIT conlimit  
| [ ENCRYPTED ] PASSWORD 'password' | PASSWORD NUL  
| VALID UNTIL 'timestamp'  
| IN ROLE role_name [, ...]  
| IN GROUP role_name [, ...]  
| ROLE role_name [, ...]  
| ADMIN role_name [, ...]  
| USER role_name [, ...]  
| SYSID uid
```

The screenshot shows a PostgreSQL management tool interface. On the left, a tree view shows the database structure: Servers (1) > PostgreSQL 16 > Databases (5) > Login/Group Roles (16). The 'joe' role is highlighted under 'Login/Group Roles (16)'. A red arrow points to the 'joe' role. On the right, the 'Query' tab shows the SQL command: `CREATE ROLE Joe WITH SUPERUSER CREATEDB CREATEROLE INHERIT LOGIN CONNECTION LIMIT 5 PASSWORD 'joepassword';`. The 'Messages' tab shows the output: 'CREATE ROLE' and 'Query returned successfully in 53 msec.'.

SUPERUSER: Gir superbrukerrettigheter til rollen.
CREATEDB: Tillater rollen å opprette databaser.
CREATEROLE: Tillater rollen å opprette andre roller.
INHERIT: Tillater rollen å arve rettigheter fra roller den er medlem av.
LOGIN: Tillater rollen å logge inn.
CONNECTION LIMIT 5: Begrenser antall samtidige tilkoblinger for rollen til 5.
PASSORD 'password': Angir passordet for rollen.

Eksempel på Oppretting av Brukerrolle

The screenshot shows a database management interface. On the left, a tree view under 'Servers (1)' > 'PostgreSQL 16' > 'Login/Group Roles (17)' has 'alex' selected, indicated by a red arrow. The main pane shows a SQL query editor with the following code:

```
1 CREATE ROLE alex WITH
2 NOSUPERUSER
3 NOCREATEDB
4 NOCREATEROLE
5 NOINHERIT
6 LOGIN
7 CONNECTION LIMIT 1
8 PASSWORD 'alexpassword';
```

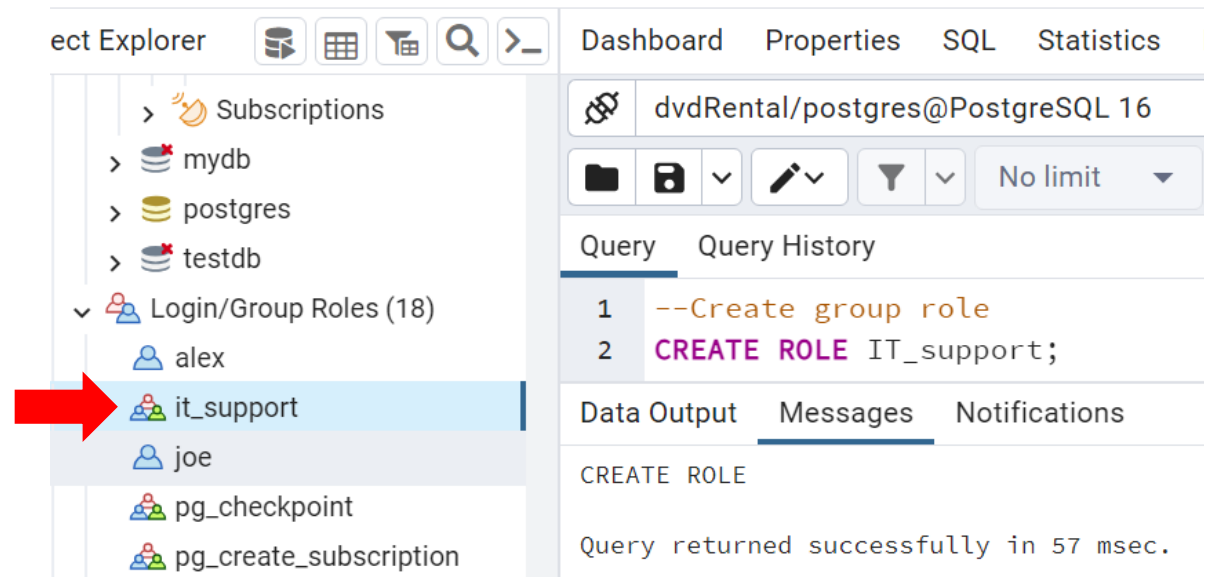
Below the query editor, the 'Messages' tab shows the output: 'CREATE ROLE' and 'Query returned successfully in 59 msec.'

NOINHERIT: Deaktiverer arv av rettigheter fra roller den er medlem av.

CONNECTION LIMIT 1: Setter maksimalt antall samtidige tilkoblinger for rollen til 1.

Gruppe Roller

- I PostgreSQL er en grupperolle en rolle som fungerer som en beholder for andre individuelle roller **for å forenkle tillatelsesadministrasjon.**
- I stedet for å gi rettigheter til individuelle roller, kan du gruppere disse rollene i en gruppe, **gi rettigheter til en grupperolle**, og alle medlemmene av den grupperollen vil arve disse rettighetene.



Administrere Privilegier

- GRANT: gi et privilegium til en bruker
- WITH GRANT OPTION: mottakeren kan i sin tur gi privilegiet til en annen bruker.
- REVOKE: ta bort et privilegium fra en bruker.

Query	Query History
1	--Grant insert privilege to all users on table film
2	GRANT INSERT ON film TO PUBLIC;

GRANT Varianter

- GRANT-kommandoen har to grunnleggende varianter:
 - Den gir rettigheter til et databaseobjekt (tabell, visning, sekvens, database, funksjon, prosedyrespråk osv.).
 - Den gir medlemskap i en rolle.

Query	Query History
1	--Grant privilege to individual role
2	GRANT DELETE ON inventory, payment, rental, film TO Alex;
Data Output	Messages
GRANT	
	Query returned successfully in 34 msec.

Query	Query History
1	--Grant all privileges to group role
2	GRANT ALL PRIVILEGES ON DATABASE "dvdRental" TO IT_support;
Data Output	Messages
GRANT	
	Query returned successfully in 32 msec.

Query	Query History
1	--Add role Joe to IT_support group role
2	GRANT IT_support TO Joe;
Data Output	Messages
GRANT ROLE	
	Query returned successfully in 68 msec.

Liste Roller

- Måter å liste roller og vist informasjon om roller på.

```
dvdRental=# \du
```

List of roles		
Role name	Attributes	
alex	No inheritance 1 connection	+
it_support	Cannot login	
joe	Superuser, Create role, Create DB 5 connections	+
postgres	Superuser, Create role, Create DB, Replication, Bypass RLS	

```
dvdRental=# \du Joe
```

List of roles		
Role name	Attributes	
joe	Superuser, Create role, Create DB 5 connections	+

```
dvdRental=# \du Alex
```

List of roles		
Role name	Attributes	
alex	No inheritance 1 connection	+

Liste Privilegier

```
dvdRental=# \z rental
```

			Access privileges		
Schema	Name	Type	Access privileges	Column privileges	Policies
public	rental	table	postgres=arwdDxt/postgres+ alex=d/postgres		

```
=xxxx -- privileges granted to PUBLIC
uname=xxxx -- privileges granted to a user
group gname=xxxx -- privileges granted to a group
```

```

r -- SELECT ("read")
w -- UPDATE ("write")
a -- INSERT ("append")
d -- DELETE
R -- RULE
x -- REFERENCES
t -- TRIGGER
X -- EXECUTE
U -- USAGE
C -- CREATE
T -- TEMPORARY
arwdRxt -- ALL PRIVILEGES (for tables)
* -- grant option for preceding privilege

/yyyy -- user who granted this privilege
```

D -- TRUNCATE

Hva er Forskjellen?

- La oss si at vi opprettet rollen «Silje». Hvis vi gir henne privilegium nr. 1, hva ville hun kunne gjøre?
- Hvis vi gir henne privilegium nr. 2, hva ville hun kunne gjøre?
- Hvis vi gir henne privilegium nr. 1 og deretter tilbakekaller alle privilegier på alle tabellene fra henne, hva ville hun kunne gjøre?

Query	Query History
1	CREATE ROLE silje;

Query	Query History
1	ALTER ROLE silje
2	LOGIN
3	PASSWORD 'siljepassword';

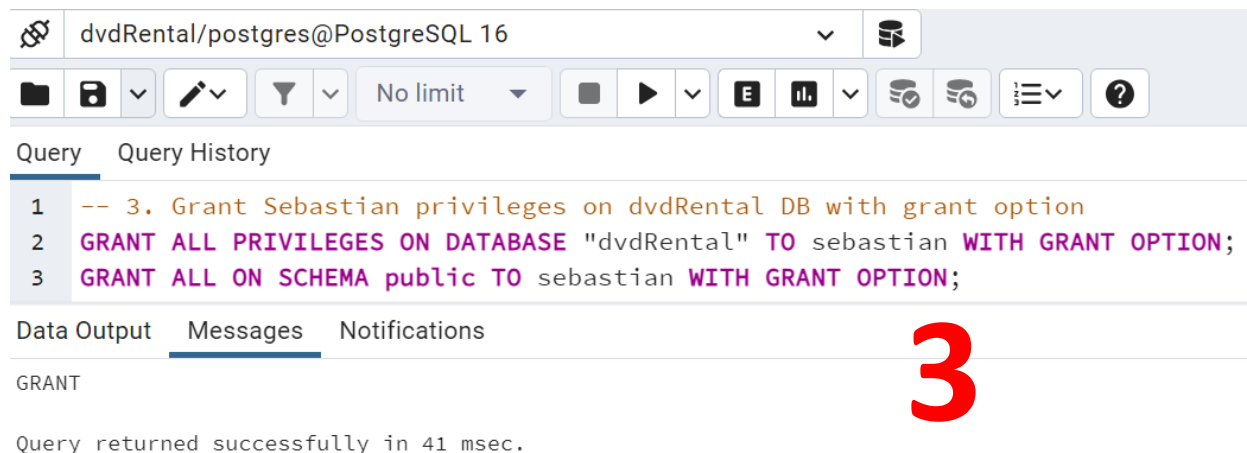
Query	Query History
1	-- Privilege #1
2	GRANT ALL ON SCHEMA public TO silje;
3	
4	-- Privilege #2
5	GRANT ALL ON ALL TABLES in SCHEMA public TO silje;

Query	Query History
1	REVOKE ALL
2	ON ALL TABLES IN SCHEMA public
3	FROM silje;

Grant Privilege With Grant Option

- Hvis «WITH GRANT OPTION» er spesifisert, kan mottakeren av rettigheten gi den til andre.
- Uten «WITH GRANT OPTION» kan ikke mottakeren gjøre det.
- WITH GRANT OPTION kan ikke gis til PUBLIC.

```
2 -- 1. Create admin group role
3 CREATE ROLE admins;
4
5 -- 2. Create superuser and grant it membership in admins group role at once
6 CREATE ROLE Sebastian WITH
7     SUPERUSER
8     CREATEDB
9     CREATEROLE
10    INHERIT
11    LOGIN
12    REPLICATION
13    CONNECTION LIMIT -1
14    PASSWORD 'adminpassword'
15    IN GROUP admins;
```



The screenshot shows a PostgreSQL client window titled 'dvdRental/postgres@PostgreSQL 16'. The interface includes a toolbar with icons for file operations, query execution, and settings. Below the toolbar, the 'Query' tab is active, displaying three lines of SQL code: a comment, a GRANT statement for the 'Sebastian' role, and a GRANT statement for the 'public' schema. The 'Messages' tab is also visible, showing the output 'GRANT' and a confirmation message: 'Query returned successfully in 41 msec.'

```
1 -- 3. Grant Sebastian privileges on dvdRental DB with grant option
2 GRANT ALL PRIVILEGES ON DATABASE "dvdRental" TO sebastian WITH GRANT OPTION;
3 GRANT ALL ON SCHEMA public TO sebastian WITH GRANT OPTION;
```

Data Output Messages Notifications

GRANT

Query returned successfully in 41 msec.

Revoke

```
REVOKE [ GRANT OPTION FOR ]
  { { SELECT | INSERT | UPDATE | DELETE | RULE | REFERENCES | TRIGGER }
    [,...] | ALL [ PRIVILEGES ] }
  ON [ TABLE ] tablename [, ...]
  FROM { username | GROUP groupname | PUBLIC } [, ...]
  [ CASCADE | RESTRICT ]

REVOKE [ GRANT OPTION FOR ]
  { { CREATE | TEMPORARY | TEMP } [,...] | ALL [ PRIVILEGES ] }
  ON DATABASE dbname [, ...]
  FROM { username | GROUP groupname | PUBLIC } [, ...]
  [ CASCADE | RESTRICT ]

REVOKE [ GRANT OPTION FOR ]
  { EXECUTE | ALL [ PRIVILEGES ] }
  ON FUNCTION funcname ( [ [ argmode ] [ argname ] argtype [, ...] ] ) [, ...]
  FROM { username | GROUP groupname | PUBLIC } [, ...]
  [ CASCADE | RESTRICT ]
```

```
REVOKE [ GRANT OPTION FOR ]
  { { CREATE | USAGE } [,...] | ALL [ PRIVILEGES ] }
  ON SCHEMA schemaname [, ...]
  FROM { username | GROUP groupname | PUBLIC } [, ...]
  [ CASCADE | RESTRICT ]

REVOKE [ GRANT OPTION FOR ]
  { CREATE | ALL [ PRIVILEGES ] }
  ON TABLESPACE tablespacename [, ...]
  FROM { username | GROUP groupname | PUBLIC } [, ...]
  [ CASCADE | RESTRICT ]

REVOKE [ ADMIN OPTION FOR ]
  role [, ...] FROM username [, ...]
  [ CASCADE | RESTRICT ]
```

Revoke

- Den brukes til enten å tilbakekalle medlemskap av en individuell brukerrolle fra en grupperolle eller tilbakekalle tilgangsrettigheter fra en individuell bruker, grupperolle eller PUBLIC (alle brukere).
- Enhver bestemt rolle vil ha summen av rettigheter gitt direkte til den, rettigheter gitt til enhver rolle den er medlem av, og rettigheter gitt til PUBLIC.
- For eksempel betyr det ikke nødvendigvis at REVOKE av SELECT-rettighet fra PUBLIC at alle roller har mistet SELECT-rettigheten på objektet: de som har fått den tildelt direkte eller via en annen rolle, vil fortsatt ha den.

Revoke membership

Query	Query History
1	--REVOKE [group role name] FROM [individual user role name];
2	REVOKE IT_support FROM Alex;

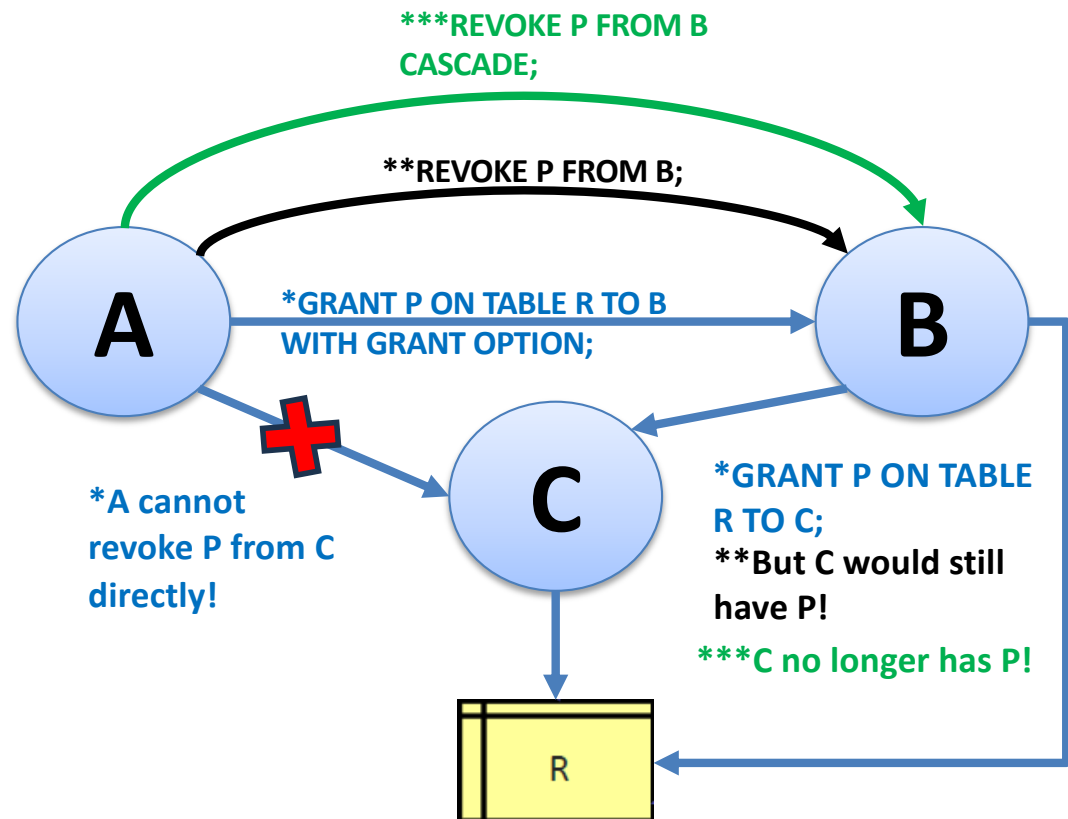
Revoke access privilege

Query	Query History
1	REVOKE DELETE ON inventory FROM alex;

Query	Query History
1	GRANT ALL PRIVILEGES ON SCHEMA public TO PUBLIC;
2	REVOKE ALL PRIVILEGES ON SCHEMA public FROM PUBLIC;

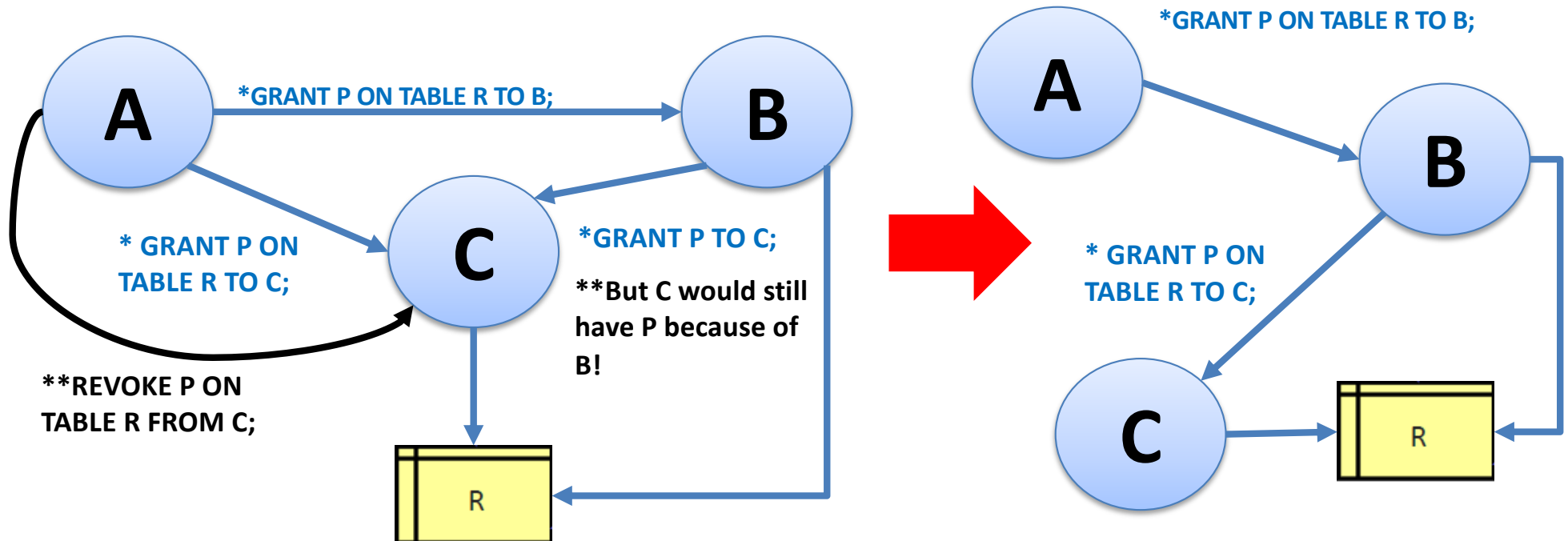
Revoke

- Å gi privilegier ved hjelp av «WITH GRANT OPTION» skaper kompleksitet ved tilbakekalling av privilegier!
- Siden alle privilegier kommer fra objektieren (muligens indirekte via kjeder av tildelingsopsjoner), er det mulig for en superbruker å REVOKE alle privilegier, men dette kan kreve bruk av CASCADE. Hvis for eksempel bruker A har gitt et privilegium (P) WITH GRANT OPTION til bruker B, og bruker B igjen har gitt det til bruker C, kan ikke bruker A REVOKE privilegiet direkte fra C.
- I stedet kan bruker A REVOKE «WITH GRANT OPTION» fra bruker B og bruke CASCADE-alternativet slik at privilegiet igjen REVOKES fra bruker C.



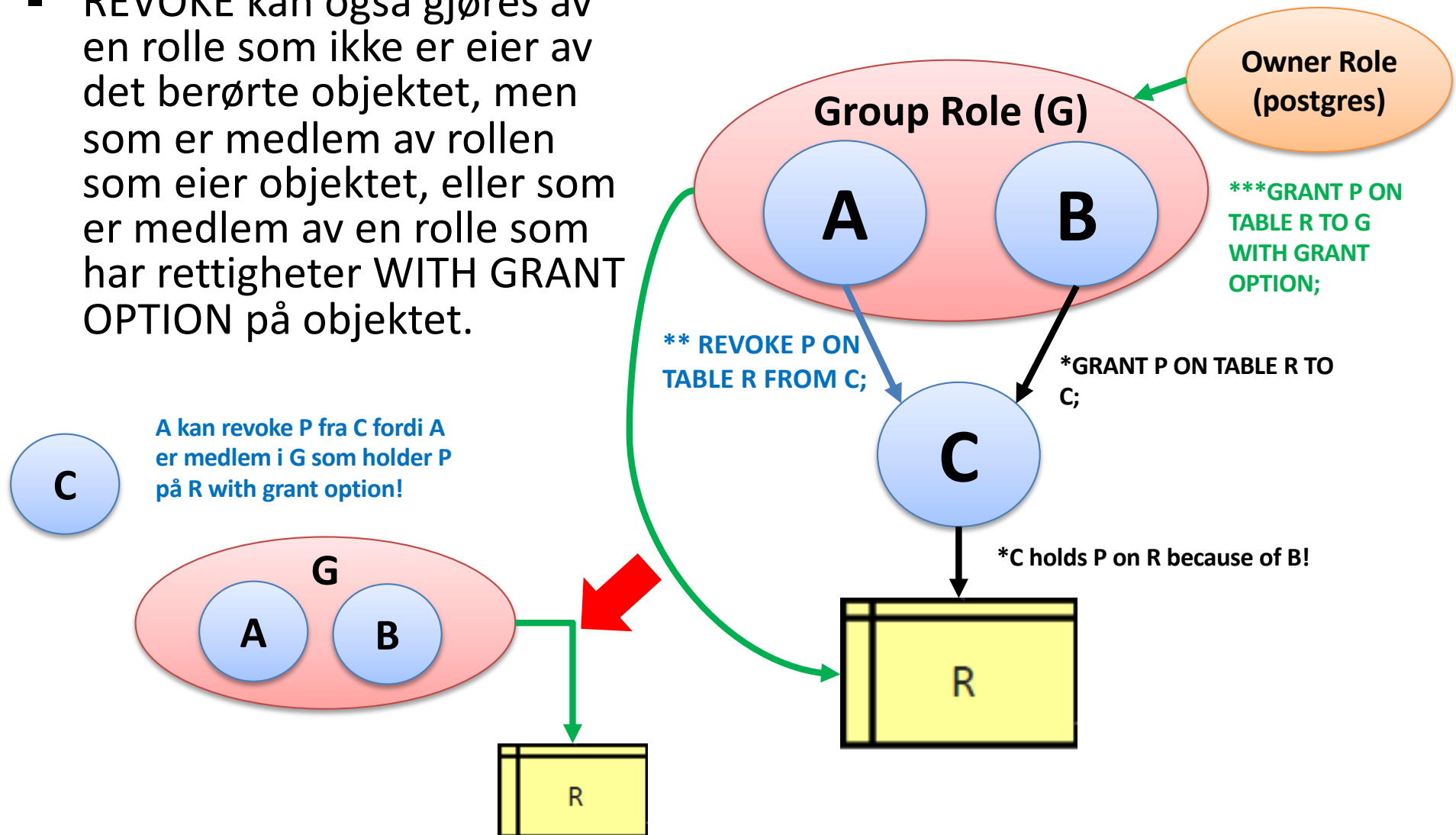
Revoke

- Et annet eksempel, hvis både A og B har gitt C samme privilegium, kan A REVOKE sitt eget privilegium, men ikke Bs privilegium, slik at C fortsatt vil ha privilegiet.



Revoke

- REVOKE kan også gjøres av en rolle som ikke er eier av det berørte objektet, men som er medlem av rollen som eier objektet, eller som er medlem av en rolle som har rettigheter WITH GRANT OPTION på objektet.



Vedlikehold av Brukerroller

- `SUPERUSER` | `NOSUPERUSER` – determine if the role is a `superuser` or not.
- `CREATEDB` | `NOCREATEDB` – allow the role to create new databases.
- `CREATEROLE` | `NOCREATEROLE` – allow the role to create or change roles.
- `INHERIT` | `NOINHERIT` – determine if the role inherits the privileges of roles of which it is a member.
- `LOGIN` | `NOLOGIN` – allow the role to log in.
- `REPLICATION` | `NOREPLICATION` – determine if the role is a replication role.
- `BYPASSRLS` | `NOBYPASSRLS` – determine if the role is to bypass the row-level security (RLS) policy.
- `CONNECTION LIMIT limit` – specify the number of concurrent connections a role can make, -1 means unlimited.
- `PASSWORD 'password'` | `PASSWORD NULL` – change the role's password.
- `VALID UNTIL 'timestamp'` – set the date and time after which the role's password is no longer valid.

Eksempel:

Query Query History

1 `ALTER ROLE Alex VALID UNTIL '2024-04-09';`

Data Output Messages Notifications

ALTER ROLE

Query returned successfully in 35 msec.

dvdRental=# \du Alex

List of roles		
Role name	Attributes	
alex	No inheritance	+
	1 connection	+
	Password valid until 2024-04-09 00:00:00+02	

Vedlikehold av Brukerroller

- Alter: rename user role.
- Kommando: `ALTER ROLE [role_old_name] TO [role_new_name];`
- En superbruker kan gi nytt navn til enhver rolle.
- En rolle som har CREATEROLE-privilegiet kan gi nytt navn til roller uten superbrukerrettigheter.
- Hvis du bruker en rolle til å logge på PostgreSQL-databaseserveren og gir den nytt navn i gjeldende økt, vil du få en feilmelding.
 - Du må koble til PostgreSQL-databaseserveren med en annen rolle for å gi nytt navn til den rollen.

Vedlikehold av Brukerroller

- Drop: DROP ROLE [IF EXISTS] [role name];
- Som «postgres»-superbruker kan du prøve kommandoen: DROP ROLE Alex;
- For å fjerne en superbrukerrolle må du være superbruker, men for å fjerne ikke-superbrukerroller må du ha CREATEROLE-rettigheten.
- Når du fjerner en rolle som eier et objekt i en database, vil PostgreSQL gi en feilmelding. I dette tilfellet må du ta to trinn:
 - Først må du enten fjerne databaseobjektene som eies av rollen ved å bruke DROP OWNED-setningen, eller tilordne eierskapet av databaseobjektene til en annen rolle, REASSIGN OWNED.
 - Deretter må du tilbakekalle eventuelle rettigheter som er gitt til rollen.

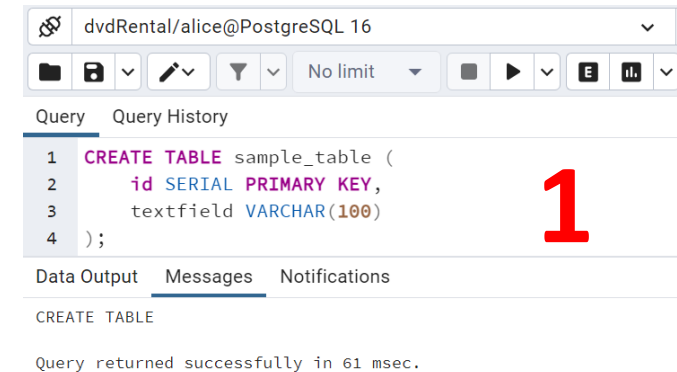
```
-- execute these statements in the database that contains
-- the object owned by the target role
REASSIGN OWNED BY target_role TO another_role;

DROP OWNED BY target_role;

-- drop the role
DROP ROLE target_role;
```

Vedlikehold av Brukerroller

- Drop:
 - Eksempel:
 - Som “alice”, prøv å opprette sample_table, etterpå logg av.
 - Logg inn igjen som “postgres” og prøv å droppe “alice” rolle.

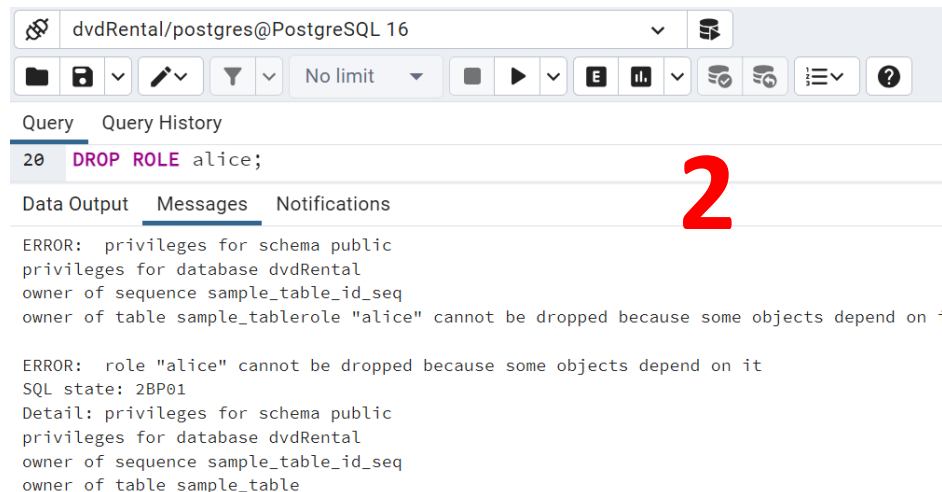


The screenshot shows a PostgreSQL client window titled 'dvdRental/alice@PostgreSQL 16'. The query editor contains the following SQL code:

```
1 CREATE TABLE sample_table (  
2     id SERIAL PRIMARY KEY,  
3     textfield VARCHAR(100)  
4 );
```

The query is executed successfully, as indicated by the message: 'Query returned successfully in 61 msec.'

1



The screenshot shows a PostgreSQL client window titled 'dvdRental/postgres@PostgreSQL 16'. The query editor contains the following SQL code:

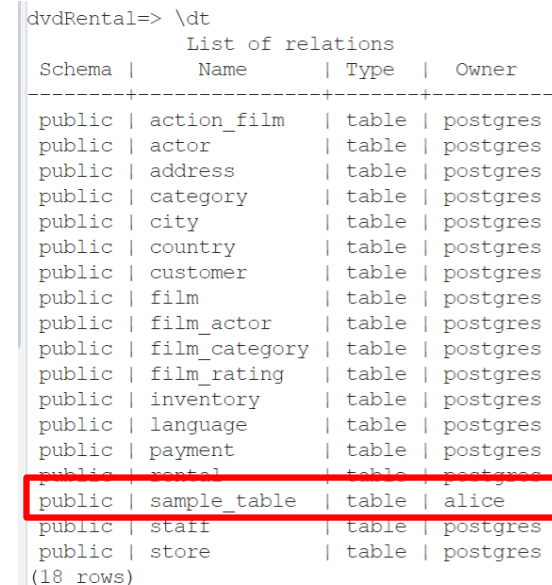
```
20 DROP ROLE alice;
```

The query is executed, but an error is returned:

```
ERROR: privileges for schema public  
privileges for database dvdRental  
owner of sequence sample_table_id_seq  
owner of table sample_table "alice" cannot be dropped because some objects depend on it  
  
ERROR: role "alice" cannot be dropped because some objects depend on it  
SQL state: 2BP01  
Detail: privileges for schema public  
privileges for database dvdRental  
owner of sequence sample_table_id_seq  
owner of table sample_table
```

2

Løsning?
Reassign, drop
ownership,
and drop role



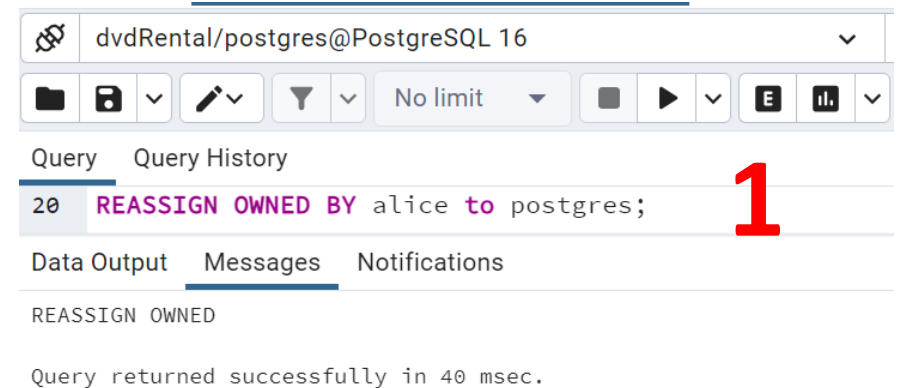
The screenshot shows the output of the command 'dvdRental=> \dt' in a PostgreSQL client window. The output is a table listing the relations in the database:

Schema	Name	Type	Owner
public	action_film	table	postgres
public	actor	table	postgres
public	address	table	postgres
public	category	table	postgres
public	city	table	postgres
public	country	table	postgres
public	customer	table	postgres
public	film	table	postgres
public	film_actor	table	postgres
public	film_category	table	postgres
public	film_rating	table	postgres
public	inventory	table	postgres
public	language	table	postgres
public	payment	table	postgres
public	rental	table	postgres
public	sample_table	table	alice
public	staff	table	postgres
public	store	table	postgres

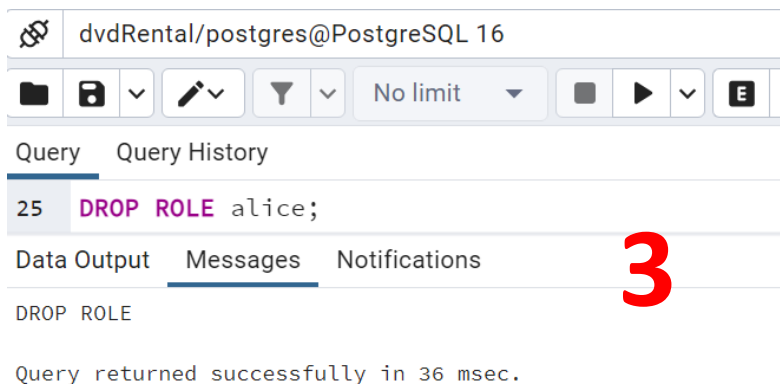
(18 rows)

Vedlikehold av Brukerroller

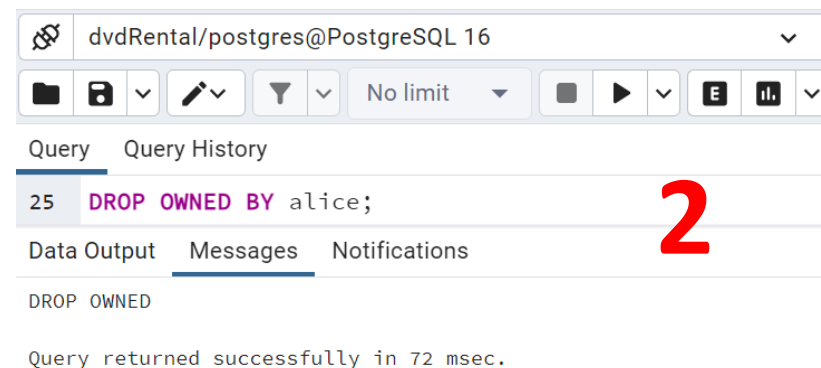
- Drop:
 - Example:
 - Som “postgres”, omfordel eide objekter av “alice” til deg selv.
 - Drop eierskap av objektene fra “alice”.
 - Til slutt, drop rolle “alice”.



The screenshot shows a PostgreSQL client window titled 'dvdRental/postgres@PostgreSQL 16'. The 'Query' tab is active, displaying the query: `20 REASSIGN OWNED BY alice to postgres;`. The 'Messages' tab is also visible, showing the response: `REASSIGN OWNED`. The status bar at the bottom indicates: 'Query returned successfully in 40 msec.' A large red number '1' is overlaid on the right side of the query text.



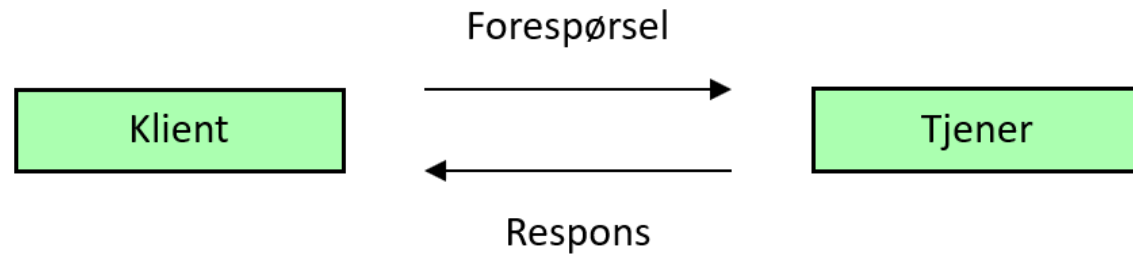
The screenshot shows a PostgreSQL client window titled 'dvdRental/postgres@PostgreSQL 16'. The 'Query' tab is active, displaying the query: `25 DROP ROLE alice;`. The 'Messages' tab is also visible, showing the response: `DROP ROLE`. The status bar at the bottom indicates: 'Query returned successfully in 36 msec.' A large red number '3' is overlaid on the right side of the query text.



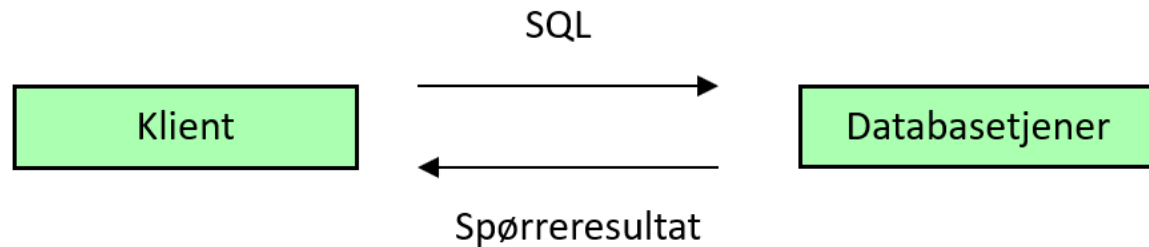
The screenshot shows a PostgreSQL client window titled 'dvdRental/postgres@PostgreSQL 16'. The 'Query' tab is active, displaying the query: `25 DROP OWNED BY alice;`. The 'Messages' tab is also visible, showing the response: `DROP OWNED`. The status bar at the bottom indicates: 'Query returned successfully in 72 msec.' A large red number '2' is overlaid on the right side of the query text.

Klient/tjener-arkitektur

- Generelt



- Databaser



- Klienter og tjener i nettverk



Skyløsninger

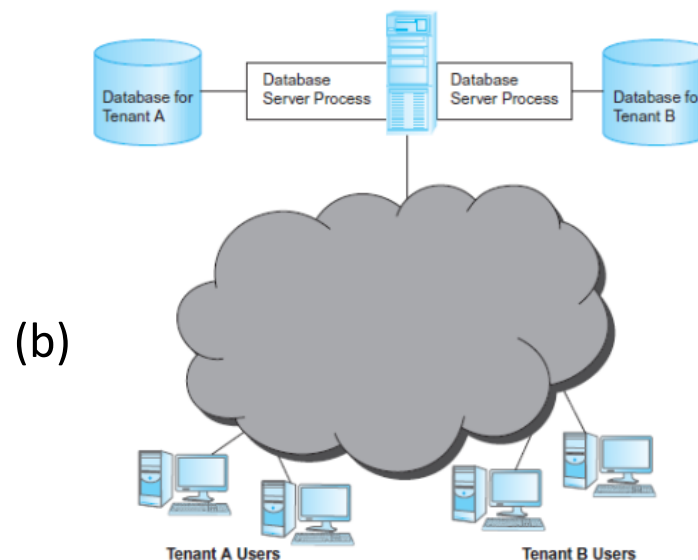
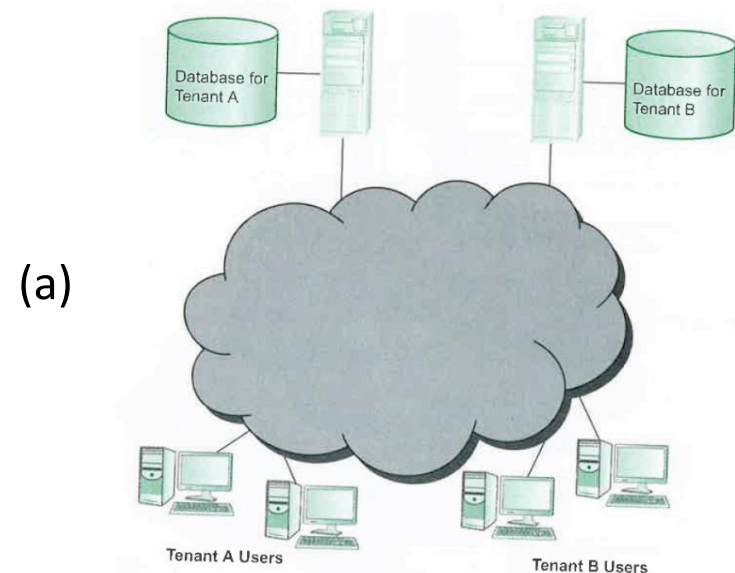
- **IaaS** (Infrastructure as a Service) er en basistjeneste der man får tilgang på en eller flere maskiner, men må selv installere operativsystem og programvare. Eksempel: Amazon Web Services.
- **PaaS** (Platform as a Service) er en mer omfattende tjeneste enn IaaS, der man i tillegg til maskiner og operativsystem også får et ferdig utviklingsmiljø i skyen, f.eks. i form av en Linux-tjener satt opp med en MySQL database.
- **SaaS** (Software as a Service) er en ytterligere utvidet tjeneste i forhold til PaaS og kan betraktes som en måte å tilby programvare på. Eksempel: Google Apps og Office365.

Databaser i skyen

- **Skydatabase** (cloud database) er en database som blir lagret i skyen, og kan i utgangspunktet være basert på både IaaS, PaaS og SaaS.
- **DBaaS** (DataBase as a Service) er en SaaS-løsning for databaser, der man typisk både oppretter, administrerer og bruker databasen via en webapplikasjon.
- **Azure SQL Database** er enkelt sagt «SQL Server i skyen» og kan fås som en DBaaS-løsning.

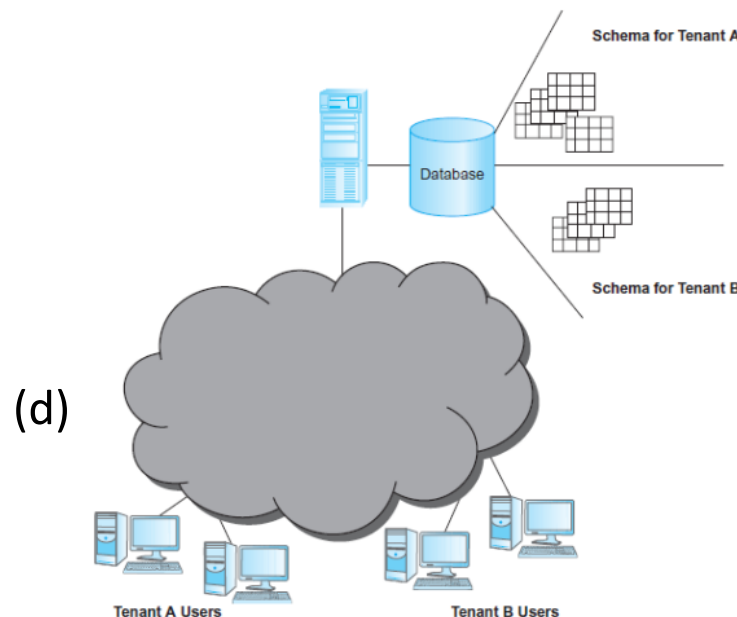
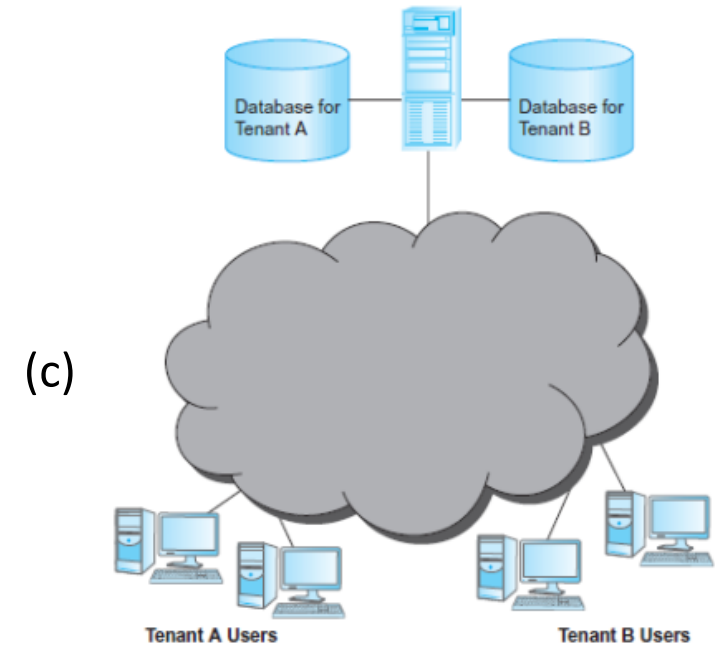
Varianter av Sky-baserte Database Løsninger

- Hver tenant har egen server og egen database (a).
- Serveren er delt, men hver tenant har egen database server prosess og egen database (b).



Varianter av Sky-baserte Database Løsninger

- Serveren er delt, men hver tenant har egen database (c).
- Serveren og databasen er delt, men hver tenant har eget schema (d).



Varianter av Sky-baserte Database Løsninger

- Serveren, databasen og schema er delt.

